



Diseñado por: Cuarto Informática 2026

Documento: Hoja de Trabajo 4

Nombre de actividad: Formulario de registro

Tema: POO, JDBC y componentes de datos en JavaFX

Objetivo:

Fecha de entrega: Semana 7 - del 20 al 24 de julio

Punteo: 20

Proyecto Integrador de bimestre parte 1

Instrucciones:

Objetivos de Aprendizaje

El estudiante deberá demostrar capacidad para:

- **Analizar y Diseñar:** Modelar una tabla de base de datos relacional eficiente para la gestión de usuarios.
- **Arquitectura de Software:** Implementar el patrón MVC (Modelo-Vista-Controlador) para desacoplar la lógica de negocio, la interfaz gráfica y el acceso a datos.
- **Persistencia:** Utilizar la API JDBC para establecer conexiones, realizar inserciones (Create) y consultas (Read) en MySQL.
- **Integración:** Conectar módulos previamente aislados (Login) con nuevos módulos (Registro) bajo un mismo ecosistema de datos.

Descripción del Desafío

El estudiante debe evolucionar su sistema de autenticación anterior. La "base de datos simulada" (listas en memoria) debe ser eliminada y reemplazada por una conexión real a MySQL.

Además, se debe desarrollar un Módulo de Registro de Usuarios. El sistema final debe permitir que un usuario nuevo se registre mediante un formulario, sus datos se guarden en la base de datos, y posteriormente pueda iniciar sesión con esas credenciales en la ventana de Login creada anteriormente.

El Reto de Análisis:

No se proveerá el código SQL de la tabla ni la estructura de las clases. El estudiante debe definir qué tipos de datos usar en MySQL y cómo diseñar las clases Java para que el patrón MVC se cumpla estrictamente.

Requerimientos de Análisis y Base de Datos (Backend)

1. Diseño del Esquema (MySQL):
 - o El estudiante debe diseñar una tabla (ej. users o usuarios) que sea capaz de almacenar la información necesaria.
 - o Debe definir correctamente las *Primary Keys*, restricciones de unicidad (para que no existan dos nombres de usuario iguales) y tipos de datos adecuados para textos y correos.
 - o Entregable: Script .sql con la creación de la base de datos y la tabla.
2. Capa del Modelo (Data Layer):
 - o Debe existir una clase que represente la entidad (POJO/Bean) con sus respectivos atributos encapsulados.



- o Debe implementar una clase separada encargada exclusivamente de la comunicación con la BD (Patrón DAO o Repository). Esta clase debe manejar:
 - La conexión (usando el Driver JDBC).
 - La sentencia SQL para insertar un nuevo registro.
 - La sentencia SQL para validar si un usuario existe (Login).

Requerimientos de Interfaz

1. Formulario de Registro:
 - o Debe mantener la línea gráfica "Undecorated" y personalizada de la actividad anterior (coherencia visual).
 - o Campos requeridos mínimos: *Nombre Completo, Nombre de Usuario, Correo Electrónico, Contraseña, Confirmar Contraseña*.
 - o Debe incluir un botón para "Registrar" y un botón/enlace para "Volver al Login".
2. Integración con Login:
 - o La ventana de Login existente debe modificarse. Ya no verificará contra un ArrayList, sino que invocará al Controlador/Modelo para consultar a la base de datos real.
 - o Se debe agregar un botón o etiqueta en el Login que diga "*¿No tienes cuenta? Regístrate aquí*", que navegue al formulario de registro.

Requerimientos de Arquitectura (MVC)

El código debe estar estrictamente organizado en paquetes (packages):

- com.alumno.proyecto.model: Contendrá la lógica de datos, la clase de conexión y la entidad Usuario.
- com.alumno.proyecto.view: Contendrá las clases gráficas (JavaFX puro) del Login y del Registro. La vista NO debe contener código SQL ni lógica de negocio compleja.
- com.alumno.proyecto.controller: Contendrá las clases que reciben los eventos de los botones (ActionListeners). El controlador recibe los datos de la Vista, los valida y los envía al Modelo.

Validaciones y Reglas de Negocio

El estudiante debe programar la lógica para asegurar la integridad de los datos:

1. Campos Vacíos: No permitir registros incompletos.
2. Confirmación de Password: Validar que el campo "Contraseña" y "Confirmar Contraseña" sean idénticos.
3. Unicidad: El sistema debe impedir que se registre un usuario si el *username* o el *email* ya existen en la base de datos (manejo de excepciones SQL o validación previa).



Rúbrica de Evaluación (20 Puntos)

Criterio	Nivel 1 (0 pts)	Nivel 2 (1.25 pts)	Nivel 3 (2.5 pts)	Nivel 4 (3.75 pts)	Nivel 5 (5 pts)
Arquitectura MVC (Organización) 5 pts	Todo el código en una sola clase o paquete. SQL dentro de la UI.	Intento de separación, pero la Vista sigue teniendo lógica de negocio o SQL.	Paquetes definidos (M-V-C) pero con dependencias cruzadas incorrectas.	Separación correcta. La Vista solo muestra, el Modelo solo procesa datos.	Implementación MVC profesional. Bajo acoplamiento y alta cohesión entre capas.
Persistencia (MySQL) (Implementación) 5 pts	No conecta a BD o usa listas en memoria.	Conecta pero falla al insertar o consultar datos.	Realiza operaciones CRUD básicas pero sin manejo de PreparedStatement (vulnerable).	Conexión estable, consultas funcionales y uso de PreparedStatement.	Manejo robusto de JDBC, cierre de recursos (conn, stmt) y gestión eficiente de conexiones.
Análisis y Funcionalidad (Lógica) 5 pts	El registro no funciona o permite duplicados fatales.	Funciona parcialmente, pero permite datos inválidos (vacíos, passwords distintos).	Valida campos básicos, pero falla al manejar usuarios duplicados en la BD.	Sistema robusto: Valida integridad de datos y gestiona errores de BD correctamente.	Análisis superior: Tipos de datos SQL óptimos y validaciones completas en el controlador.
Integración y UX (Interfaz) 5 pts	Las ventanas están desconectadas o no tienen diseño.	Diseño pobre o inconsistente con la actividad anterior. Navegación falla.	Diseño coherente. Navegación Login <-> Registro funcional.	Experiencia fluida. Diseño atractivo y feedback visual al usuario (éxito/error).	Interfaz profesional personalizada, transiciones suaves y mensajes de error amigables al usuario.